Unix & Shell Programming

Module 5 – The VI Editor

Question & Answer Notes

BCA Semester 6 · MAKAUT

Q1. What is the VI Editor? Why is it used?

VI (pronounced "vee-eye") stands for **Visual Editor**. It is a powerful, screen-based text editor available on almost every UNIX and Linux system. It was created by **Bill Joy** in 1976 at the University of California, Berkeley.

The modern improved version is called **Vim** (Vi IMproved), which adds syntax highlighting, undo history, and many other features.

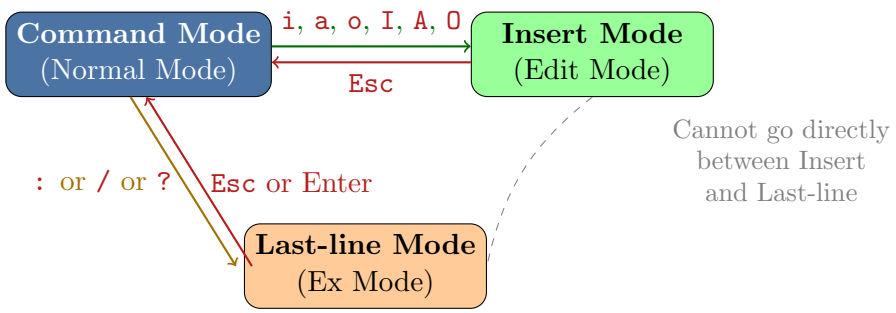
Why is VI used?

1. **Available everywhere:** VI is pre-installed on virtually every UNIX/Linux system. Even on minimal servers without a GUI, VI is present.
2. **Lightweight and fast:** It uses very little memory and opens even large files instantly.
3. **No mouse needed:** VI is fully keyboard-driven. This is essential when working over SSH on remote servers.
4. **Powerful editing:** Complex find-and-replace, macros, and multi-file editing are possible.
5. **Works in terminal:** VI runs inside any terminal window – no graphical interface required.
6. **Scripting integration:** VI commands can be used non-interactively in shell scripts (`ex` mode).
7. **Industry standard:** System administrators, developers, and DevOps engineers all rely on VI for quick edits on remote machines.

> **VI vs Vim:** `vi` is the classic editor. `vim` is the improved version. On most modern Linux systems, typing `vi` actually opens `vim`. Both work the same way for basic editing.

Q2. Three Modes of VI: Command, Insert, and Last-line Mode

VI works in **three modes**. Understanding these modes is the key to using VI. Every key does something different depending on which mode you are in.



1. Command Mode (Normal Mode):

This is the **default mode** when VI opens. In this mode, every key is a command – you can move the cursor, delete, copy, paste, search, but *you cannot type text directly.*

- Press `Esc` anytime to return to Command Mode from any other mode.
- Example commands: `dd` (delete line), `yy` (copy line), `/word` (search).

2. Insert Mode (Edit Mode):

In this mode, you can **type and edit text** just like a normal text editor. The word `-- INSERT --` appears at the bottom of the screen.

- Enter from Command Mode by pressing: `i`, `a`, `o`, `I`, `A`, or `O`.
- Exit back to Command Mode by pressing `Esc`.

3. Last-line Mode (Ex Mode):

In this mode, commands are typed at the **bottom line** of the screen after a colon `:`. Used for saving, quitting, search-replace, and other powerful operations.

- Enter from Command Mode by pressing `:`, `/`, or `?`.
- Example: `:w` (save), `:q` (quit), `:s/old/new/g` (replace text).
- Press `Esc` to cancel and return to Command Mode.

Q3. Opening and Editing Files using `vi filename`

Opening a file:

```
vi myfile.txt           # open existing file, or create new file
vi file1 file2          # open multiple files
vi +10 myfile.txt       # open file and go to line 10
vi +/pattern file.txt   # open file and jump to first match of '
   pattern'
vim myfile.txt          # open with Vim (improved vi)
view myfile.txt         # open in read-only mode
```

Basic editing workflow:

Step 1: Open file: `vi myfile.txt` – starts in **Command Mode**

Step 2: Press `i` to enter **Insert Mode**

Step 3: Type your text

Step 4: Press `Esc` to return to **Command Mode**

Step 5: Type `:wq` and press `Enter` to **save and quit**

Screen layout of VI:

```
+-----------------------------------------------+
| This is the content area where text appears.  |
| Lines of your file are shown here.            |
|                                               |
| ~                                             |
| ~   (tilde ~ means empty line below file)     |
| ~                                             |
+-----------------------------------------------+
| -- INSERT --            10,5    All   |  <- status bar
+-----------------------------------------------+
```

If VI opens and nothing works – you are in Command Mode. Press `i` first to start typing. This is the most common confusion for beginners.

Q4. Cursor Movement Commands

In Command Mode, these keys move the cursor *without* using arrow keys.

Basic movement (works like arrow keys):

Key	Movement
`h`	Move **left** one character
`l`	Move **right** one character
`j`	Move **down** one line
`k`	Move **up** one line

Word movement:

Key	Movement
`w`	Move to the start of the **next word**
`b`	Move **back** to the start of the previous word
`e`	Move to the **end** of the current word
`W`	Move to next word (ignoring punctuation)
`B`	Move back a word (ignoring punctuation)

Line movement:

Key	Movement
`0`	Move to the **beginning** of the line
`^`	Move to the first non-blank character of the line
`$`	Move to the **end** of the line
`Enter`	Move to the beginning of the next line
`-`	Move to the beginning of the previous line

File / page movement:

Key	Movement
`gg`	Go to the **first line** of the file
`G`	Go to the **last line** of the file
`:N`	Go to line number N (e.g., `:25` goes to line 25)
`Ctrl+f`	Scroll **forward** one full page
`Ctrl+b`	Scroll **backward** one full page
`Ctrl+d`	Scroll **down** half page
`Ctrl+u`	Scroll **up** half page
`H`	Move to **top** of visible screen
`M`	Move to **middle** of visible screen
`L`	Move to **bottom** of visible screen

Repeating movements with a number:

```
5j      # move down 5 lines
3w      # move forward 3 words
10h     # move left 10 characters
```

Memory trick for h, j, k, l:
h = left (h is leftmost key), **l** = right (l is rightmost key)
j looks like a down arrow, **k** points upward

Q5. Text Insertion Commands

All of these are pressed in **Command Mode** and switch VI to **Insert Mode**.

Key	What it does
`i`	Insert text **before** the cursor (most commonly used)
`I`	Insert text at the **beginning** of the current line
`a`	**Append** text **after** the cursor
`A`	Append text at the **end** of the current line
`o`	Open a **new line below** the current line and enter Insert Mode
`O`	Open a **new line above** the current line and enter Insert Mode
`s`	Delete the character under cursor and enter Insert Mode
`S`	Delete the entire current line and enter Insert Mode
`r`	**Replace** one character (no mode switch, stays in Command Mode)
`R`	Enter **Replace Mode** – overwrite characters one by one

Illustration – where insertion happens:

```
       cursor
         |
Hello World
     ^
     |
  i = insert here (before cursor)
  a = insert here (after cursor)
  I = insert at start of line (before 'H')
  A = insert at end of line (after 'd')
  o = open new line below this line
  O = open new line above this line
```

Always press `Esc` after you finish typing in Insert Mode to go back to Command Mode. Forgetting to press Esc is the most common VI mistake.

Q6. Delete Commands

All delete commands work in **Command Mode**. Deleted content is stored in a buffer and can be pasted with `p`.

Command	What it deletes
`x`	Delete the **character** under the cursor
`X`	Delete the character **before** the cursor (like Backspace)
`dw`	Delete from cursor to the end of the **current word**
`db`	Delete from cursor to the beginning of the **previous word**
`dd`	Delete the **entire current line**
`D`	Delete from cursor to the **end of the line**
`d0`	Delete from cursor to the **beginning of the line**
`d$`	Same as D – delete to end of line
`dG`	Delete from current line to the **last line** of file
`dgg`	Delete from current line to the **first line** of file
`:N,Md`	Delete lines N through M (e.g., `:3,7d` deletes lines 3–7)

Using a count with delete:

```
5x      # delete 5 characters
3dd     # delete 3 lines
2dw     # delete 2 words
```

Deleted text is not gone forever! It is stored in VI's buffer. Use `p` to paste it back. This is also how you *move* text in VI: `dd` to cut, move cursor, `p` to paste.

Q7. Copy and Paste Commands

In VI, "copy" is called **yank** (y). Yanked text is placed in the buffer, then pasted with `p` or `P`.

Command	What it does
	Yank (copy) the current line
`yy`	Same as yy – yank current line
`Y`	Yank from cursor to end of current word
`yw`	Yank from cursor to end of line
`y$`	Yank from cursor to beginning of line
`y0`	Yank from current line to last line of file
`yG`	Yank lines N to M (e.g., `:2,5y`)
`:N,Myw`	**Paste** yanked/deleted text **after** the cursor / below current line
`p`	Paste yanked/deleted text **before** the cursor / above current line
`P`	

Using a count:

```
3yy      # copy 3 lines
2yw      # copy 2 words
5p       # paste 5 times
```

Move a line (cut and paste):

```
dd       # cut (delete) current line -- stored in buffer
5j       # move cursor 5 lines down
p        # paste below current line
```

Duplicate a line:

```
yy       # yank (copy) current line
p        # paste it right below -- line is now duplicated
```

Q8. Undo and Redo Commands

Command	What it does
	Undo the last change
`u`	Undo all changes on the current line
`U`	**Redo** – undo the undo (re-apply change)
`Ctrl+r`	**Repeat** the last command (very useful!)
`.`	

Examples:

```
u        # undo last action
u        # undo the action before that
u        # keep pressing u to undo more steps
Ctrl+r   # redo (undo the undo)
Ctrl+r   # redo one more step
```

```
.        # repeat last action
# Example: type 'dd' to delete a line, then press '.' to delete
    the next
# Example: type 'iHello Esc', then move and press '.' to insert
    'Hello' again
```

Vim vs vi: Classic `vi` allows only *one* level of undo. `vim` supports multiple levels of undo (unlimited with `u` key). On most modern systems you are using `vim` even when you type `vi`.

Q9. Save and Exit Commands

These commands are typed in **Last-line Mode** (after pressing `:`) except for `ZZ`.

Command	What it does
`:w`	**Save** (write) the file – stay in VI
`:w filename`	Save as a **new filename** (like Save As)
`:q`	**Quit** VI – works only if no unsaved changes
`:wq`	**Save and quit** (most commonly used)
`:wq!`	Force save and quit
`:q!`	**Quit without saving** – discard all changes
`:x`	Save and quit (same as :wq, but only writes if changed)
`ZZ`	Save and quit (shortcut – no colon needed!)
`ZQ`	Quit without saving (shortcut)
`:w!`	Force write (overwrite even read-only file, if permitted)
`:N,Mw file`	Save only lines N to M to a file

Most commonly used combinations:

```
:wq      # save the file and exit -- USE THIS MOST OFTEN
```

```
:q!      # exit WITHOUT saving -- when you made a mistake and
   want to discard
ZZ       # quick save and quit (press Shift+Z twice in Command
   Mode)
:w       # save without exiting -- useful during long editing
   sessions
```

Got stuck in VI? Press `Esc` (a few times to be safe), then type `:q!` and press Enter. This quits without saving. This is the most important command to remember!

Q10. Opening a New File using `:e filename`

The `:e` command lets you open a different file **without leaving VI**.

```
:e newfile.txt          # open a new file in the same VI window
:e /etc/passwd          # open a system file
:e!                     # reload the CURRENT file (discard
   unsaved changes)
```

Working with multiple files:

When you open VI with multiple files or use `:e`, you can switch between them:

```
vi file1.txt file2.txt # open two files at once

:n                     # move to the NEXT file
:N                     # move to the PREVIOUS file
:rew                   # rewind -- go back to first file

:args                  # list all open files
:e file3.txt           # add and switch to another file
```

Split window editing (Vim):

```
:sp filename           # split window horizontally, open file
:vsp filename          # split window vertically
Ctrl+w w               # switch between split windows
Ctrl+w q               # close current split window
```

Buffer tip: When you switch files with `:e`, VI warns you if the current file has unsaved changes. Save with `:w` first, or use `:e!` to forcefully discard changes and open the new file.

Q11. Forward and Backward Searching

Searching in VI is done in **Command Mode**. Results are highlighted and the cursor jumps to the match.

Command	Description
`/pattern`	Search **forward** (downward) for pattern
`?pattern`	Search **backward** (upward) for pattern
`n`	Go to the **next** match (same direction)
`N`	Go to the **previous** match (opposite direction)
`*`	Search forward for the **word under cursor**
`#`	Search backward for the word under cursor
`%`	Jump to the **matching bracket** `( ) [ ] { }`

Examples:

```
/hello          # search forward for "hello" -- press Enter
?world          # search backward for "world" -- press Enter
n               # jump to next occurrence of last search
N               # jump to previous occurrence
/hello\c        # case-insensitive search (add \c)
/^The           # search for lines starting with "The"
/end$           # search for lines ending with "end"
```

Enabling search highlighting in Vim:

```
:set hlsearch      # highlight all search matches
:set incsearch     # highlight match as you type (incremental)
:noh               # turn off highlighting (after you're done)
:set ignorecase    # make all searches case-insensitive
```

Search wraps around: When searching forward with `/`, VI reaches the end of the file and wraps back to the beginning to continue searching. The message `search hit BOTTOM, continuing at TOP` appears.

Q12. Text Replacement using `:s/old/new/g`

The `:s` (substitute) command replaces text. It is used in **Last-line Mode**.

Basic syntax:

`:[range]s/old/new/[flags]`

Examples – current line only (no range):

```
:s/hello/world/          # replace FIRST occurrence of 'hello'
   with 'world' on current line
:s/hello/world/g         # replace ALL occurrences on current
   line (g = global)
```

```
:s/hello/world/i          # replace, ignore case
:s/hello/world/gi         # replace all, ignore case
:s/hello/world/c          # ask for confirmation before each
   replacement
```

Examples – with line range:

```
:1,10s/old/new/g          # replace in lines 1 to 10
:5s/old/new/g             # replace in line 5 only
:.s/old/new/g             # replace in current line (. = current
   line)
:$s/old/new/g             # replace in last line ($ = last line)
:1,$s/old/new/g           # replace in entire file (same as :%s)
:%s/old/new/g             # replace in ENTIRE FILE (most common)
```

Flags summary:

Flag	Meaning
g	Global – replace all occurrences in the range (not just the first)
i	Ignore case during matching
c	Confirm – ask before each substitution
e	Suppress error if pattern not found

Q13. Global Search and Replace

Global replace means replacing text throughout the **entire file**. The standard command is:

```
%s/old_text/new_text/g
```

The % means "all lines" (line 1 to last line).

```
:%s/python/Java/g         # replace every "python" with "Java"
    in entire file
:%s/  /  /g               # replace double space with single
   space everywhere
:%s/\t/ /g                # replace all tabs with spaces
:%s/^/# /                 # add "# " at the beginning of every
    line
:%s/$/ ;/                 # add " ;" at the end of every line
:%s/^[ \t]*//             # remove leading whitespace from all
    lines
:%s/[ \t]*$//             # remove trailing whitespace from
   all lines
:%s/foo/bar/gi            # global replace, case-insensitive
```

```
:%s/foo/bar/gc              # global replace with confirmation
   prompt
```

Special characters in replacement patterns:

Symbol	Meaning
^	Beginning of line (use in search pattern)
$	End of line (use in search pattern)
.	Any single character
*	Zero or more of the preceding character
&	In replacement, refers to the matched text
\n	Newline
\t	Tab character

```
:%s/hello/[&]/g       # wraps every "hello" in brackets: [hello]
                       # & = the matched text in replacement
:%s/^/>> /            # add ">> " to beginning of every line
:%s/error/ERROR/gc    # replace "error" with "ERROR" with
   confirmation
```

Using the :g (global) command with actions:

The :g command runs a VI command on all lines matching a pattern.

```
:g/pattern/d            # delete all lines containing "pattern"
:g/^#/d                 # delete all lines starting with # (
   comments)
:g/^$/d                 # delete all empty lines
:g/error/p              # print (display) all lines containing "
   error"
:g/TODO/s/TODO/DONE/g   # on lines with "TODO", replace TODO
   with DONE
```

Quick reference – all substitution patterns:

Command	Effect
:s/a/b/	Replace first 'a' on current line
:s/a/b/g	Replace all 'a' on current line
:5,10s/a/b/g	Replace all 'a' on lines 5 to 10
:%s/a/b/g	Replace all 'a' in entire file
:%s/a/b/gc	Replace all 'a' with confirmation
:%s/a/b/gi	Replace all 'a' case-insensitively
:g/pat/d	Delete all lines matching pattern

The `c` flag is your safety net: When doing a global replace for the first time, add `c` at the end (`:%s/old/new/gc`). VI will show each match and ask `y/n/a/q` – press `y` to replace, `n` to skip, `a` to replace all remaining, `q` to quit.

— End of Module 5 Notes —